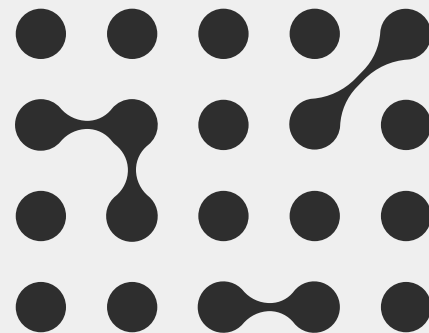


Large Scale Sorting

Micro and Macro-architecture

Wenyao Chen (z5253448)
Jackie Deng (z5160608)
Mingxuan Yu (z5261472)





Contents



1. Introduction (Wenyao)
2. Overview of Sorter Architectures (Wenyao)
3. Optimised Merge-sort Micro-Architecture (Jackie)
4. Merge-sort Macro-Architecture and Memory Management (Mingxuan)
5. Conclusion, Recent Progress and Future (Jackie)

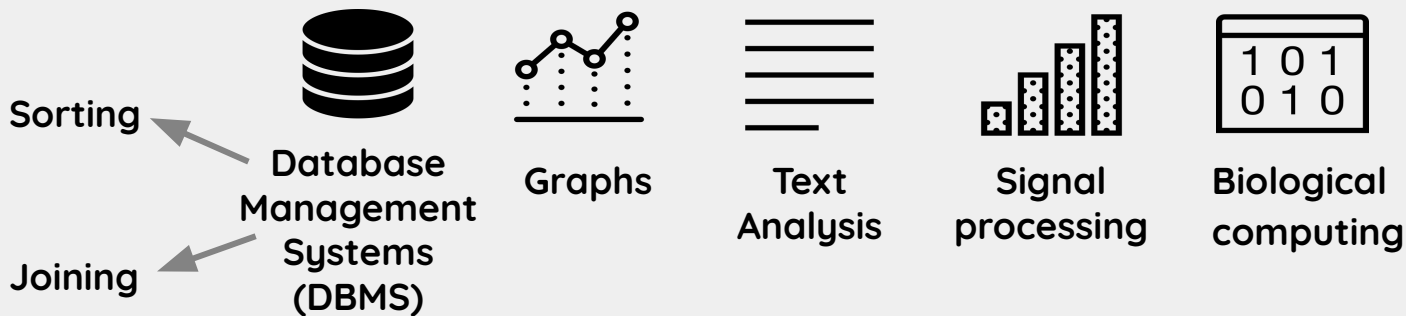
Introduction



Why Large-scale sorting important?



Sorting is a key kernel in numerous big data applications



Why Large-scale sorting important?



Bottleneck in performance:

- The resource budget allocated for sorting is often limited.
- The computational overhead and memory requirement of fast sorting is ever increasing due to the increasing size of the datasets of interest.

=> The high computational overhead of sorting has prompted research into sorting accelerators.

Why FPGA?



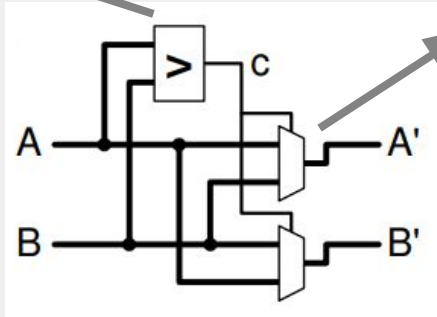
Advantages:

- Synchronous circuits is more easier to design..
- FPGAs have potential for tailored design and parallelism.
- FPGAs can be very useful as database coprocessors attached to an engine running on conventional CPUs.

Basic Sorting Elements

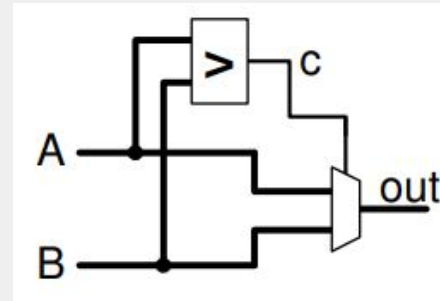


Comparator



compare-swap element

Multiplexers

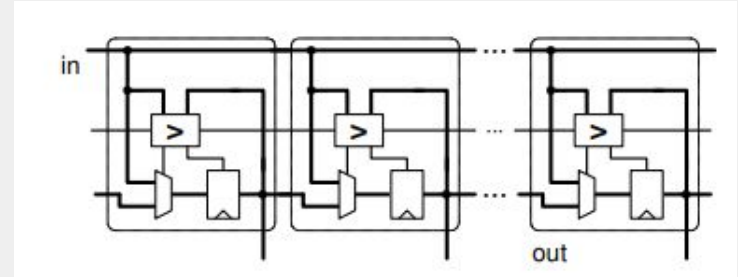
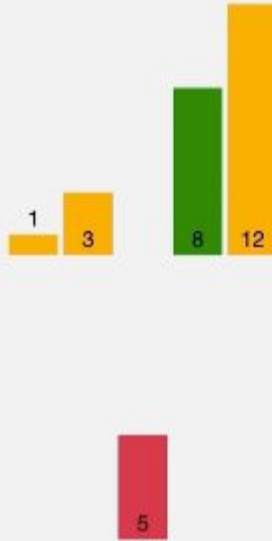


select-value element

Sorter Architectures

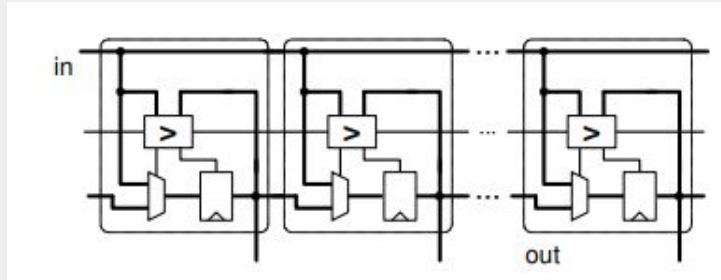


Insertion Sorter



- The resource requirements are N comparators and N registers for sorting N keys in N operation cycles.
- Time Complexity: $O(n^2)$

Insertion Sorting



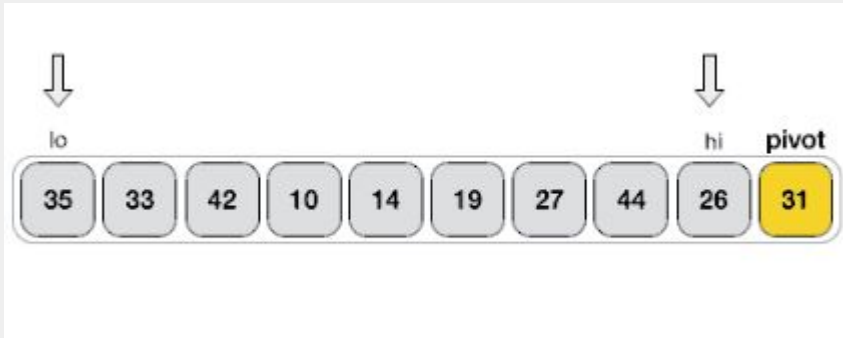
Insertion sorter

Issue:

The basic architecture of an insertion sorter would not allow fast and large implementations with many comparators because the input signal has to be broadcasted to all comparator instances.

=> we consider insertion sorting only for smaller problem sizes.

Bucket Sort

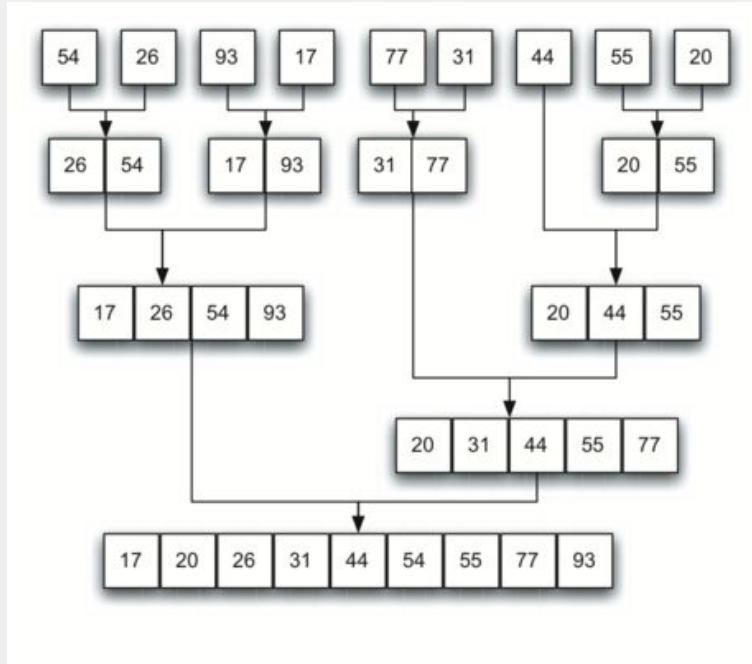


- Bucket sort follows a divide and conquer strategy by splitting the whole search problem into buckets containing fewer search keys than the original problem.
- Input stream can be stored in different regions of attached memory.
- Time Complexity (worse case): $O(n^2)$

Issue:

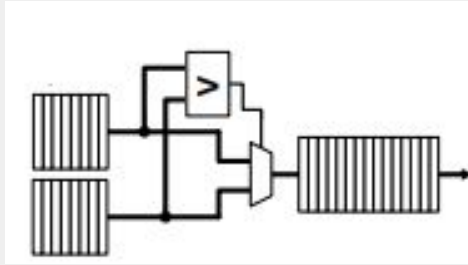
Bucket sort might perform poorly (with in the worst case quadratic time complexity)

Merge Sorter

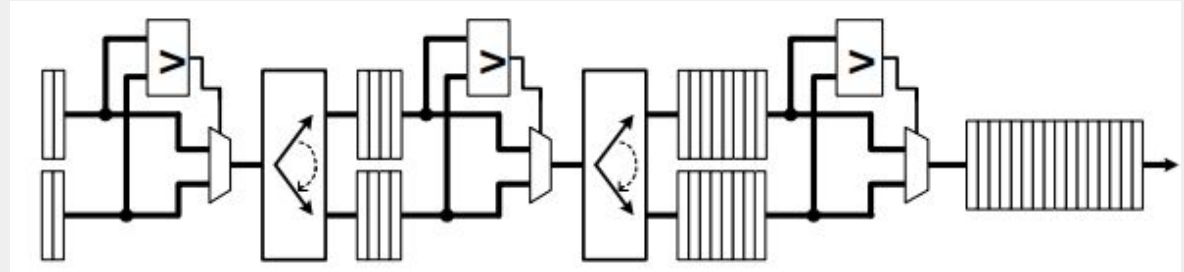


- Time Complexity: $O(n \log n)$

FIFO-Based Merge Sorter



baseline approach

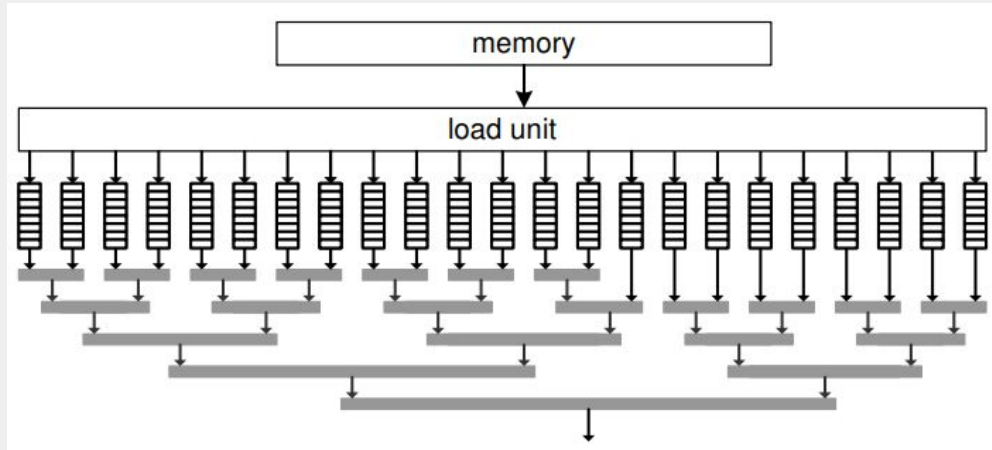


cascading FIFO-based merge sorters

Issue:

- Cost of doubling the amount of local on-chip memory in each sorter stage.

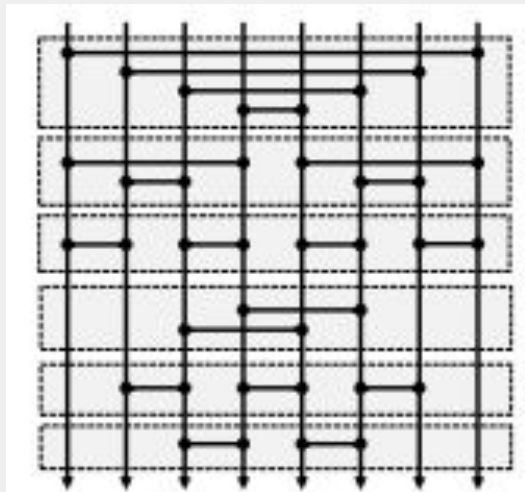
Merge Sorter Trees



Issue:

- **Backpressure flow control in the network.**

Sorting Networks



- Sorting networks are mathematical models of networks with compare-swap elements designed to sort a set of input values very fast by exploiting a high degree of parallelism.
- Common architectures include:
bitonic merge sorter networks and **even-odd networks**
- Time Complexity: **$O(\log N * \log N)$**
- Applications:
Cell Broadband Engine Architecture & GPUs

Issue:

For large problems, sorting networks require greater I/O throughput as they consume more sort keys and produce a huge amount of result data at the same time.

Brief Summary of Sorter Architectures



1. Insertion Sorter
2. Bucket Sorter
3. Merge Sorter (memory issue and flow control issue)
 - FIFO-Based
 - Tree-Based
4. Sorting Network (require large I/O throughput)

Optimised Merge-sort Micro-Architecture

2017 IEEE 25th Annual International Symposium on Field-Programmable Custom Computing Machines

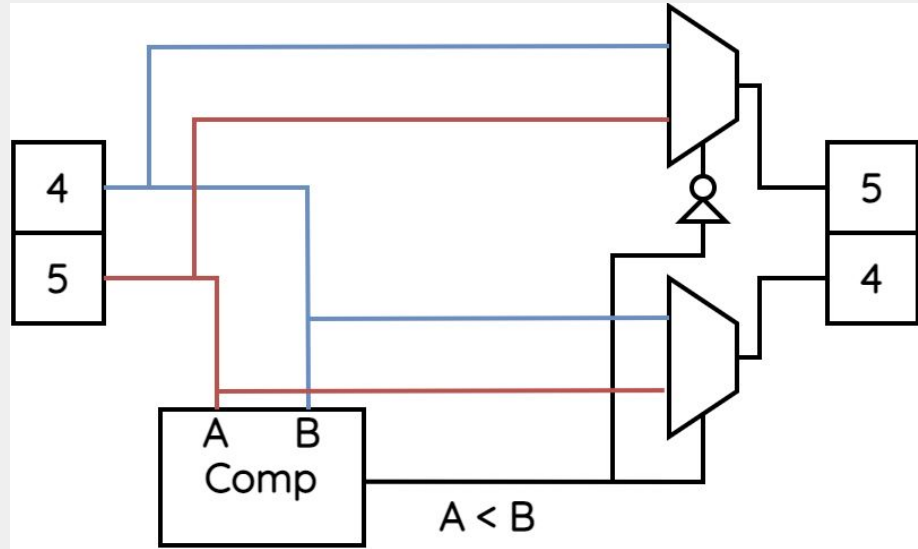
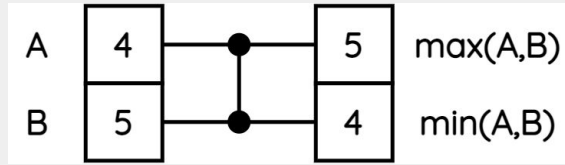
Terabyte Sort on FPGA-Accelerated Flash Storage

Sang-Woo Jun, Shuotao Xu, Arvind
Department of Electrical Engineering and Computer Science
Massachusetts Institute of Technology
mail: {wjun, shuotao, arvind}@csail.mit.edu

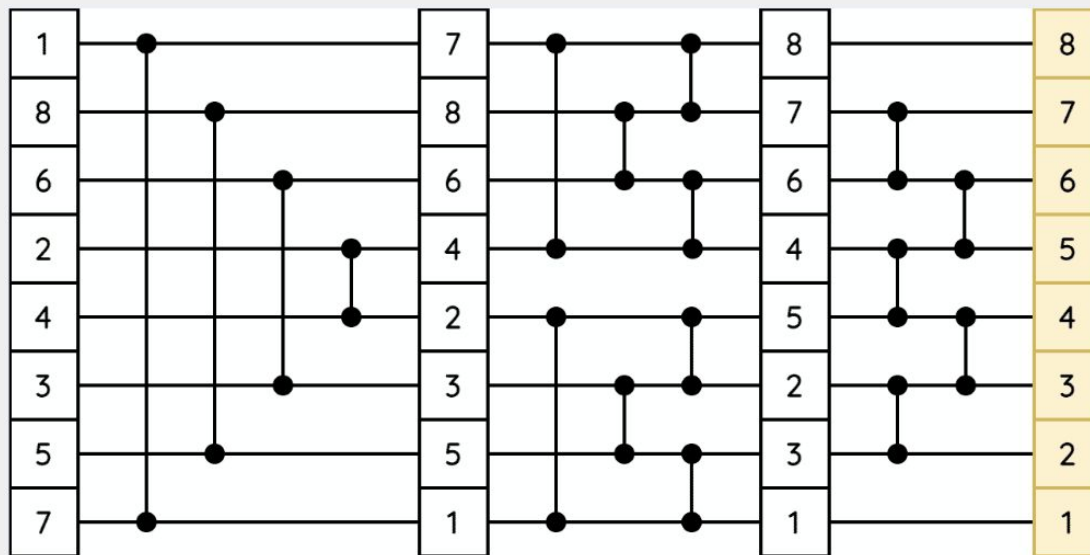
17

Abstract—Sorting is one of the most fundamental and use- machines that use a MapReduce processing platform such as

Intro to Sorting Network Diagrams



Optimal Parallel Sorting of 8 Elements



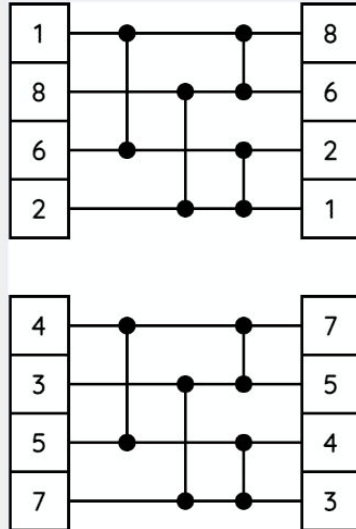
5 Cycle
Latency
(in-place)

17
'Compare-and-Swaps'

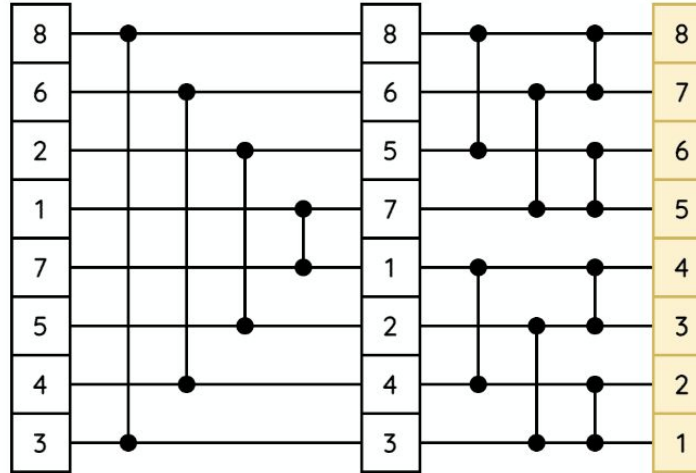
Bitonic Sorter: 8 as 2 x 4



Bitonic Sorter



Bitonic Half Cleaner



Bitonic Sorter



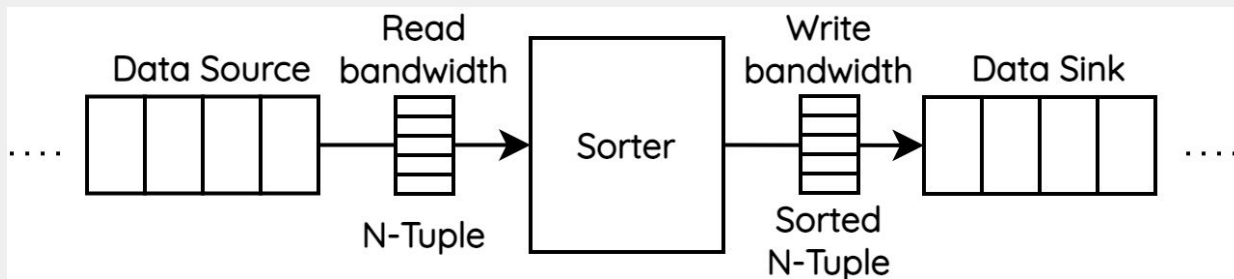
5 Cycle
Latency
(in-place)

20
'Compare-and-swaps'

Sorting by N-Tuples



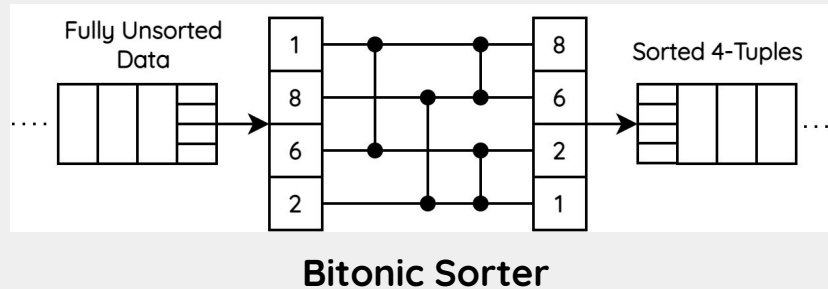
- Imagine we have data of arbitrary size -> We can't parallel sort all data
- It must be stored in secondary storage
- An optimal sorter will input values at the read band-width of the storage device
 - AND output values at the write band-width of the storage device
- An N-Tuple has $N = \text{floor}(\text{bandwidth} / \text{sizeof}(\text{element}))$ elements
- We can read and sort data as N-Tuples



Sorting N-Tuples at Input



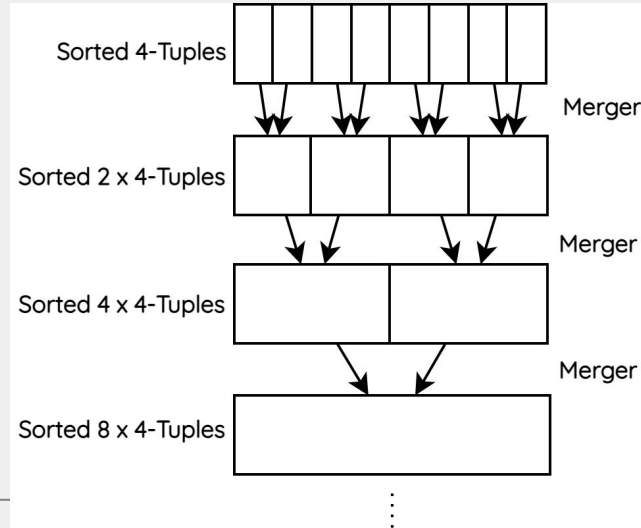
- Suppose we have a bandwidth of 64 bits / cycle and each element is 16 bits wide.
- $N = 64/16 = 4$. We will use 4-tuples.
- At the input stream, we can use the bitonic sorter to convert fully unsorted data into a stream of sorted 4-tuples



Merging Sorted N-Tuples



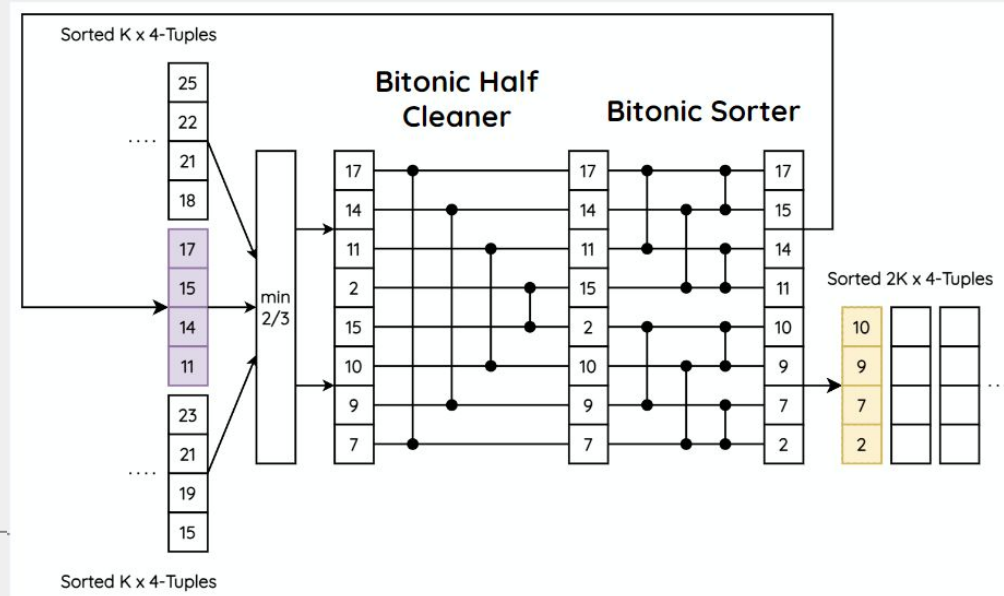
- In subsequent stages of the merge sort, we need a merger component that can take sequential N-tuples from any two $K \times N$ -Tuple Lists and output a list $2K \times N$ -tuples at one N-tuple per cycle.



Bitonic Sorter in FIFO Merge-sort



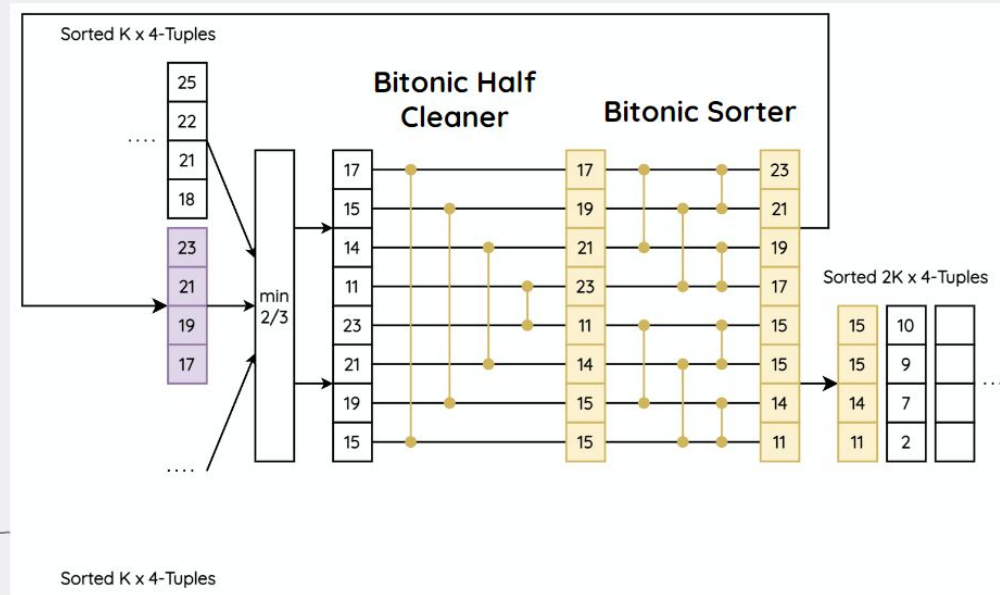
- This is where the Bitonic sorter structure comes in handy
- If sorting in ascending order, the lower half is guaranteed to be in correct order but the upper half may not be. Thus we use FEEDBACK.



Bitonic Sorter in FIFO Merge-sort



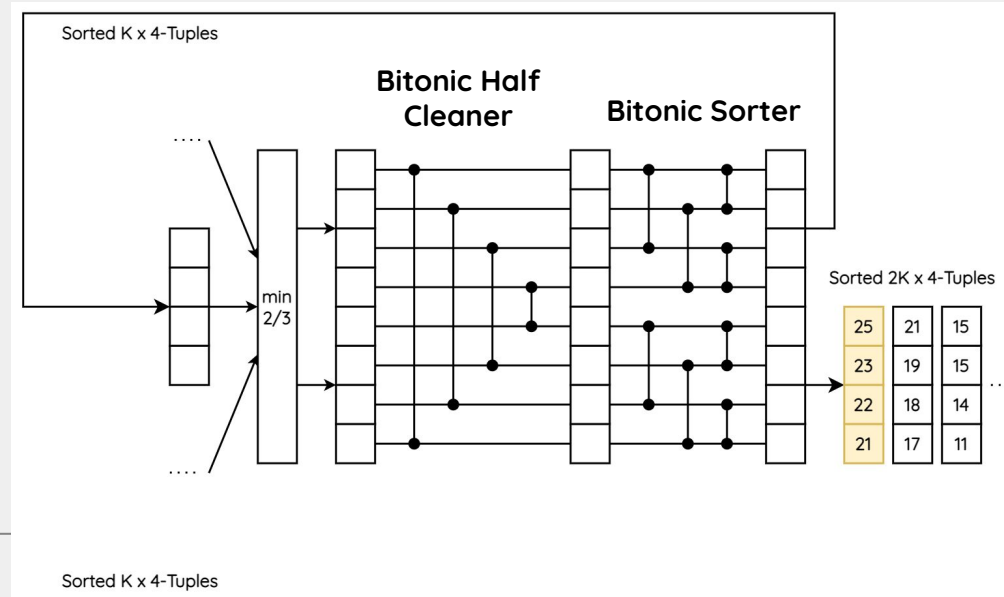
- The Feedback 4-Tuple is guaranteed to become one of the next inputs through $\min 2/3$
- The sorter completes when FIFOs are exhausted.



Bitonic Sorter in FIFO Merge-sort



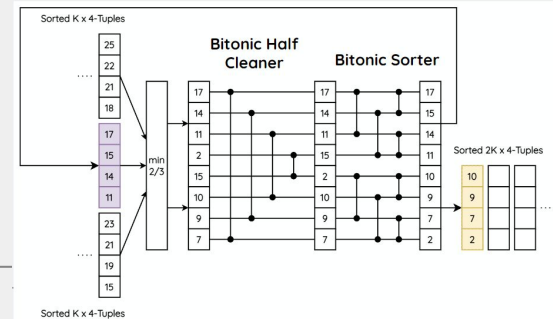
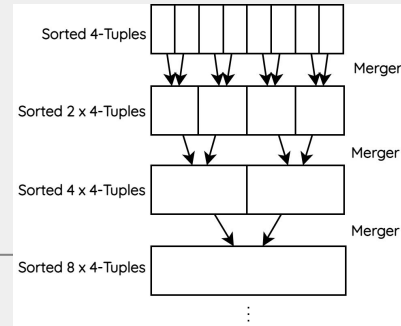
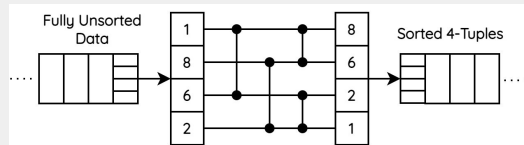
- The Feedback 4-Tuple is guaranteed to become one of the next inputs through $\min 2/3$
- The sorter completes when FIFOs are exhausted.



Recap of Micro-Architecture

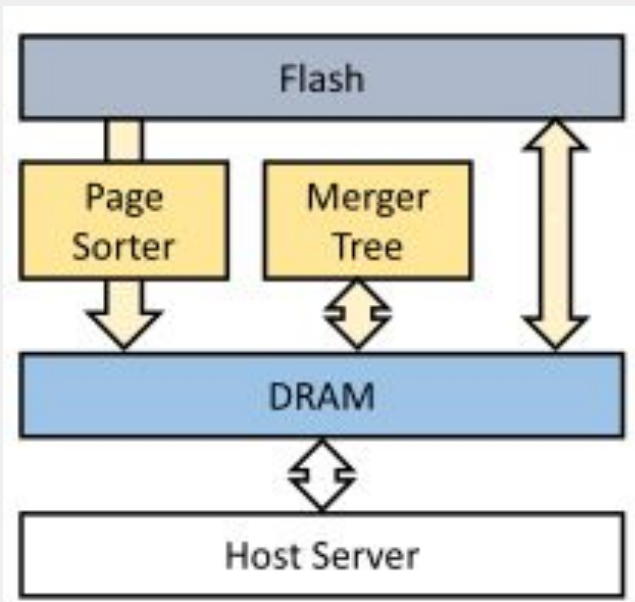


- Objective: Design a system that can take input at and produce sorted output at the data bandwidth rate (where n is fixed and known).
- **Technique #1:** Use **Parallel sorters** to sort N -tuples directly at input
- **Technique #2:** **Merge N -tuples** into longer and longer sorted sequences
- **Technique #3:** Use a **Bitonic FIFO merge-sorter** to merge $k * N$ -tuple sequences and produce a sorted N -tuple every iteration
- Jun and Xu were able to achieve a 1 cycle interval using intermediate registers and chaining, thus sorting 1 N -tuple / cycle.



Macro-Architecture





General Structure

- Terabyte sort on FPGA accelerated flash storage.

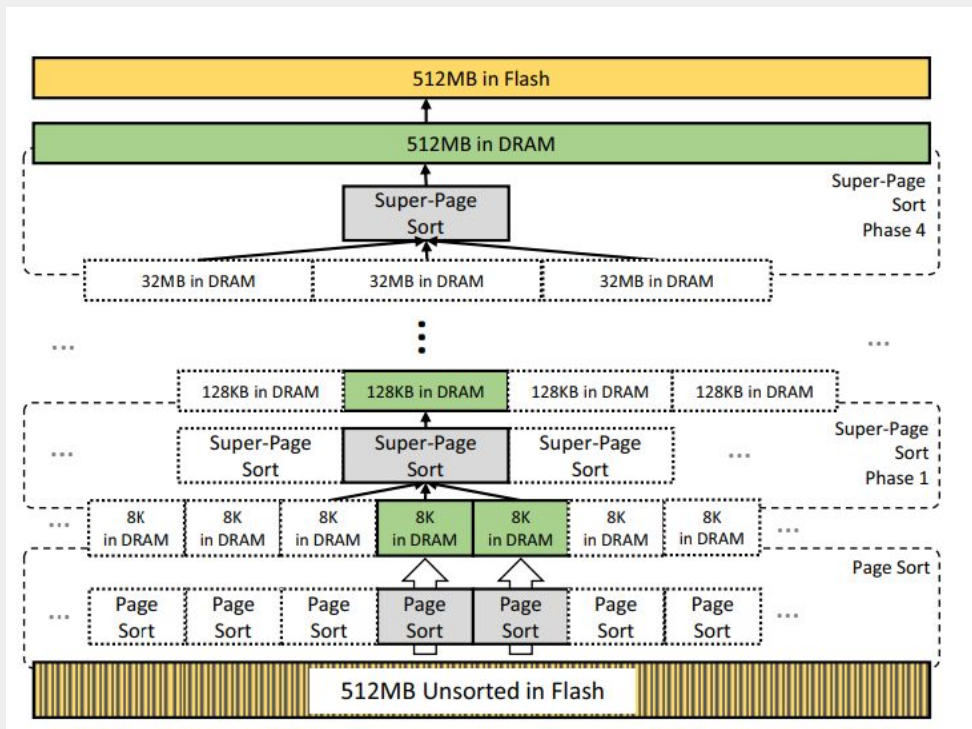


Hardware

- Custom Flash expansion card with read speed up to 2Gb/s
- Xilinx VC707 FPGA development board
- DDR3 DRAM with sequential read speed of 10GB/s



Sort into DRAM



■ Page sort

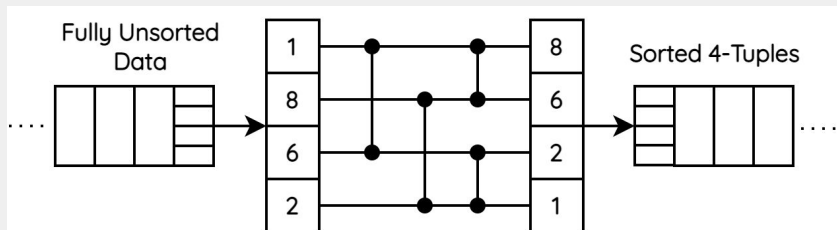
Sort random list into sorted chunk

■ Super Page sort

Merge 2 sorted chunk



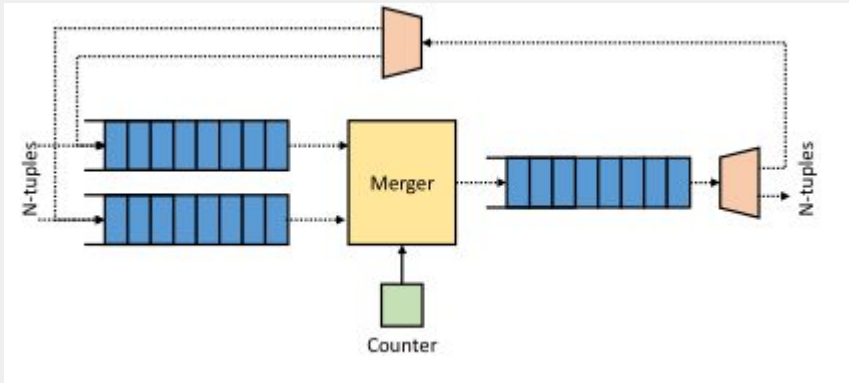
Tuple sorter



- Sorting the random input into sorted chunks



Page sorter



- Input: sorted tuple from last section
- Output 8KB chunks of sorted data

Row = DRAM page

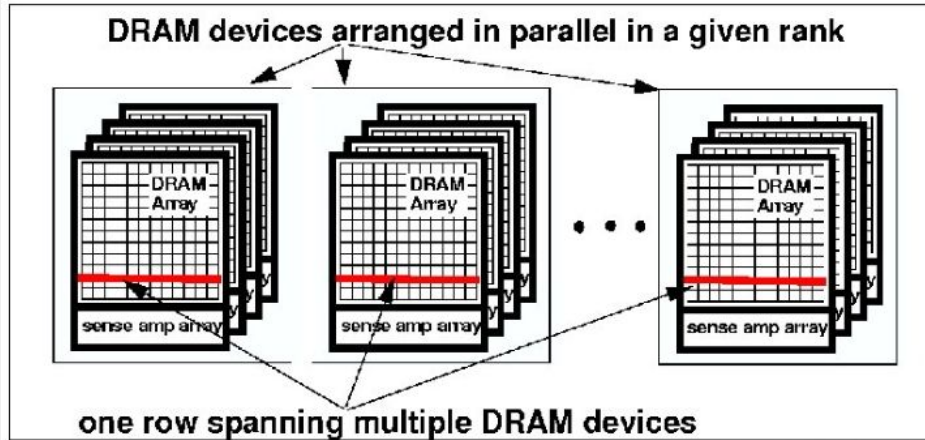
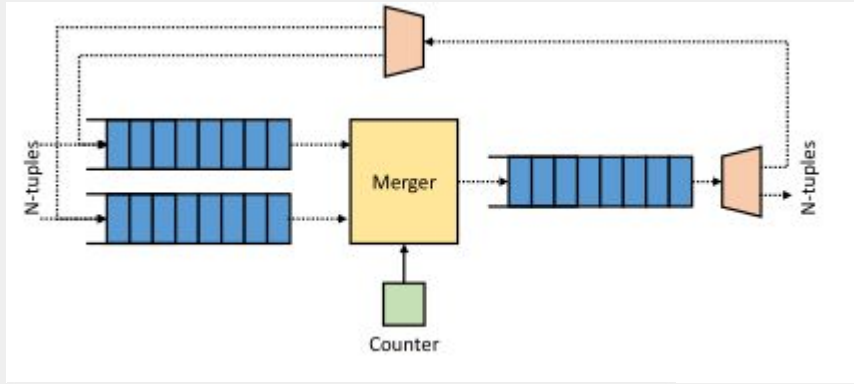


Figure 3.7: DRAM devices with 4 banks, 8192 rows per bank, 512 columns per row, and 16 bits per column.

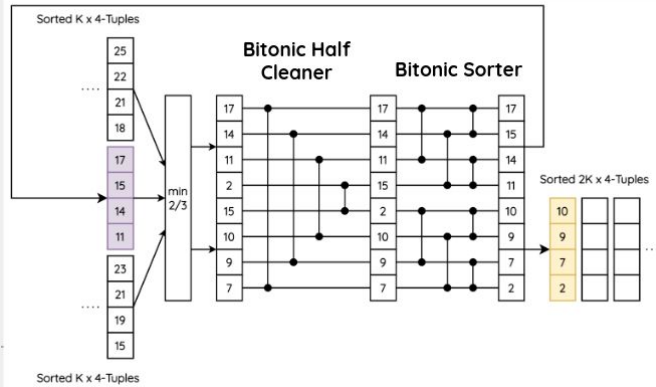
DRAM

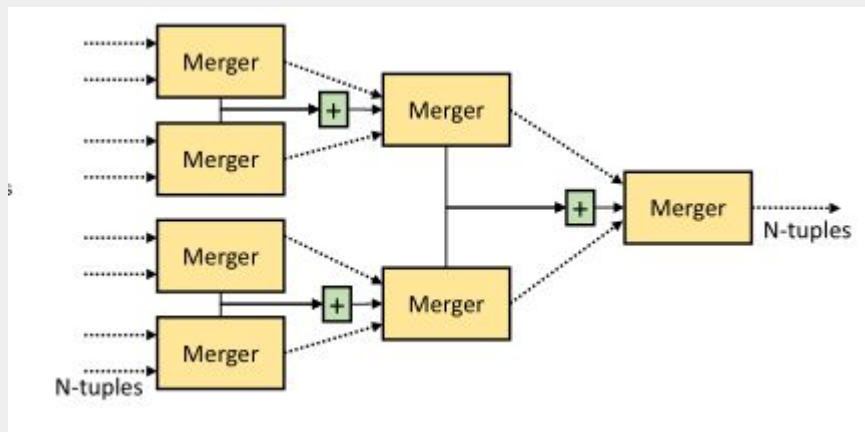
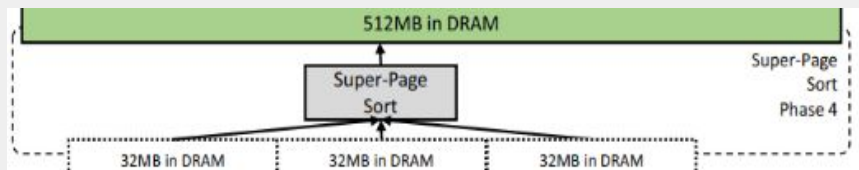
- The DRAM has 8kb banks
- 10GB/s within the banks
- Random 64-byte read 1GB/s



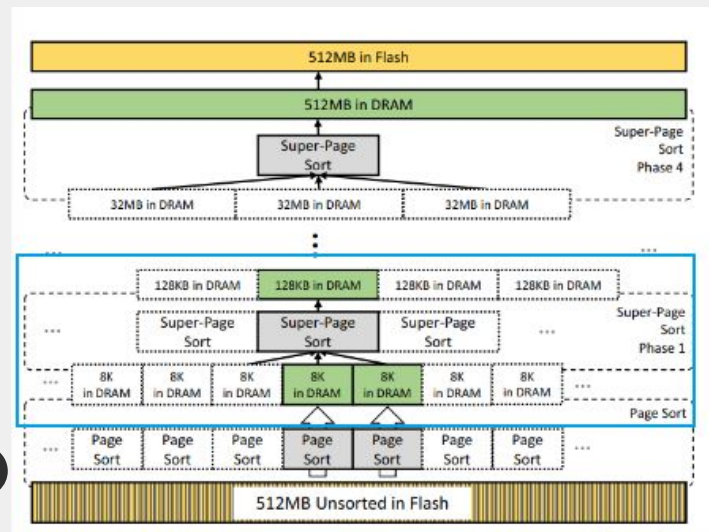
Page sorter

- Using the merger to merge 2 sorted chunks

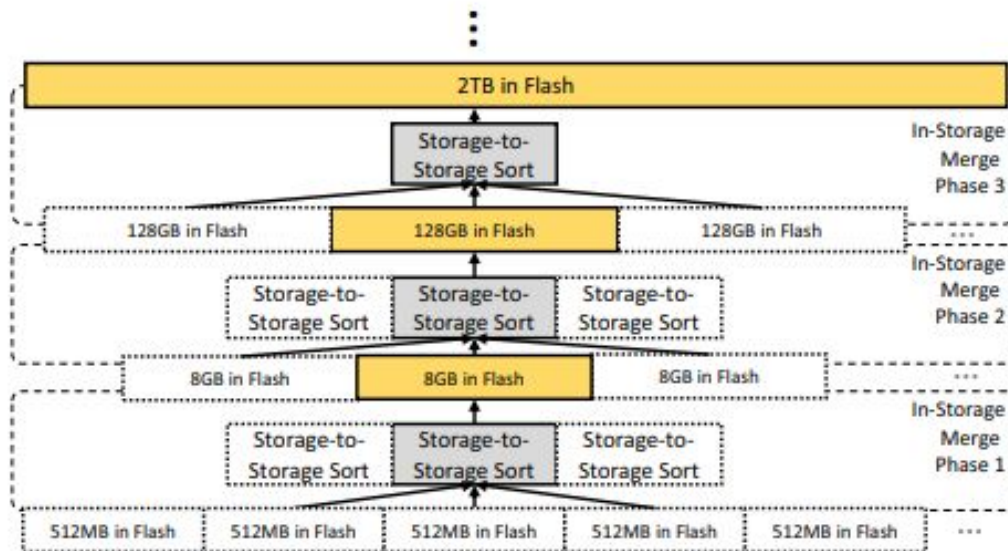




Merger Tree



Sort into DRAM



- Sorting the chunks in the flash storage
- Using BRAM as a read/write buffer

Performance

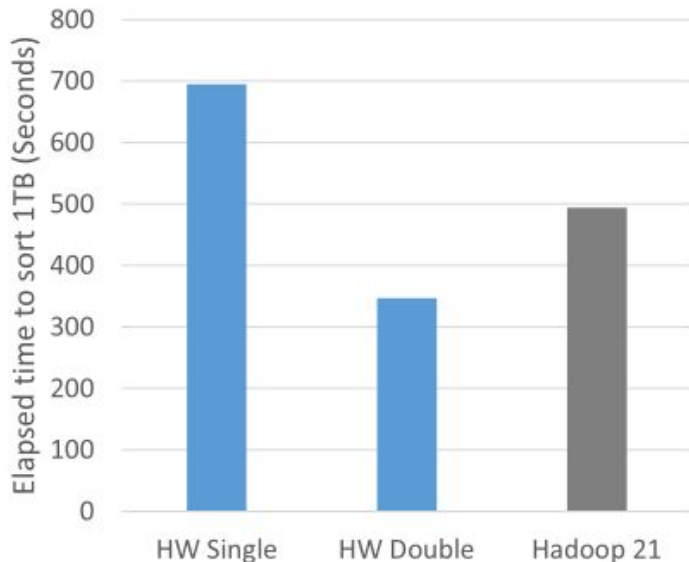
Sort Phase	Bandwidth (GB/s)	Accelerator	Required Instances
Flash Read	2.4	Page Sorter	5
In-memory	5	Merge Tree	2
Flash Write	2	Merge Tree	1
In-storage	1	Merge Tree	1

- Using multiple instance of Hardware to utilize the entire IO bandwidth
- 2390s until fully sorted

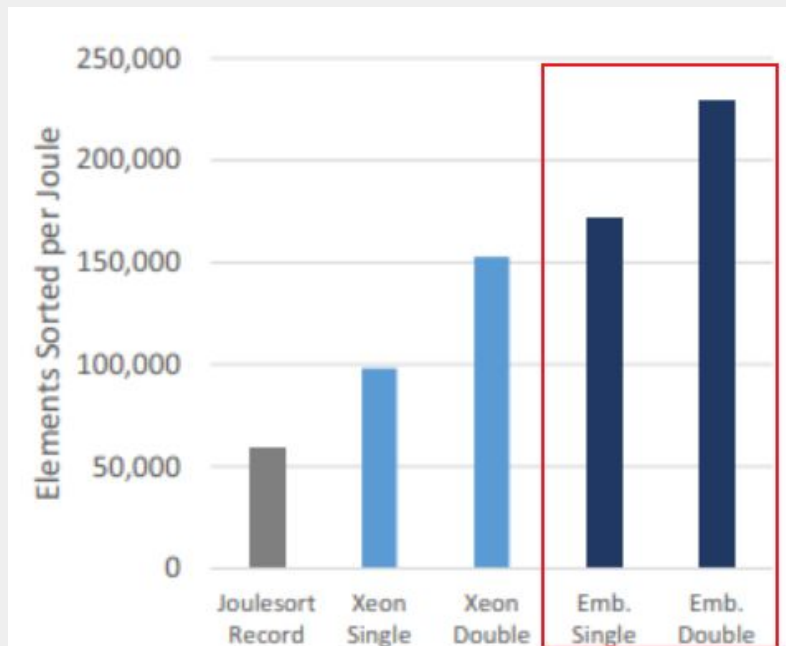
Sorted Block Size log(bytes)	Medium	Bandwidth (GB/s)	Time (s)
13	flash-DRAM	2.4	220
17	DRAM-DRAM	5	110
21	DRAM-DRAM	5	110
25	DRAM-DRAM	5	110
29	DRAM-flash	2	280
33	flash-flash	1	520
37	flash-flash	1	520
41	flash-flash	1	520
		total	2390

TABLE II
512GB SORTING PERFORMANCE BREAKDOWN BY MERGE PHASE

Performance Comparison



Power consumption



- 250000 elements sorted per joule

Conclusion



What we covered



- **Importance of large-scale sorting:** Database management, text analysis, signal processing, scientific computing...
- **Why FPGAs are a great choice for the task:** They can implement efficient parallelised sorting architectures, especially compared to CPU clusters
- **Micro-architecture for FIFO mergesort:** Sorting networks and circuit level components to sort at the rate of the data bandwidth using N-tuples
- **Macro-architecture for FIFO mergesort:** Memory hierarchy considerations when working with terabyte sized data, with the example of a flash-based system by Jun and Xu (2017) showing record-breaking performance and efficiency compared to the existing technologies

Recent Progress and Future



2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)

FANS: FPGA-Accelerated Near-Storage Sorting

Weikang Qiao*, Jihun Oh†, Licheng Guo*, Mau-Chung Frank Chang*, Jason Cong*

* University of California, Los Angeles, USA † Samsung Electronics, Hwasung, Korea

wkqiao2015@ucla.edu, ozfoot.oh@samsung.com, lcguo@cs.ucla.edu,

mfchang@ee.ucla.edu, cong@cs.ucla.edu

Abstract—Large-scale sorting is always an important yet demanding task for data center applications. In addition to powerful processing capability, high-performance sorting system requires efficient utilization of the available bandwidth of various levels in the memory hierarchy. Nowadays, with the explosive

computationally intensive and data intensive. On one hand, the sorting phase relies on a high-performance processor to sort the data. On the other hand, the merging phase moves the data

freq

TABLE VI: Comparison between FANS and [14].

	Item	[14]	FANS
Hardware setup	DRAM BW. (GB/s)	16	16
	DRAM Capacity (GB)	1	4
	Flash Read BW. (GB/s)	2.4	3
	Flash Write BW. (GB/s)	2	3
Design Config.	No. of Leaf Mergers	16	64
	Frequency (MHz)	125	250
	Sorted chunk size M (MB)	512	256
Performance	Sorting Performance (GB/s)	0.21	0.68 (3.2×)

Using Samsung SmartSSD, not Flash

[14] is Jun and Xu's work



128 GB ($2^{33} = 8.6 \times 10^9$ records) in 190 s

~ 45 million records / s

Recent Progress and Future



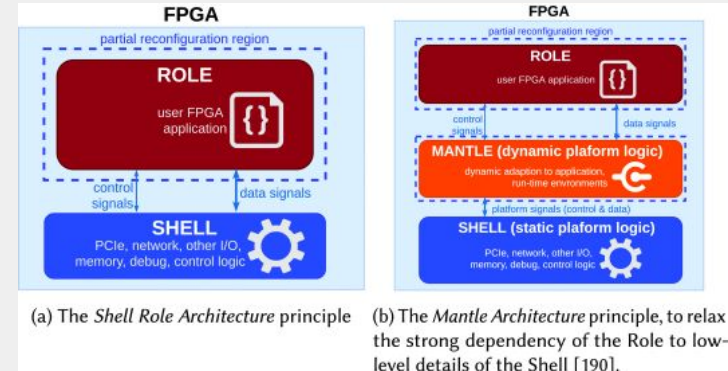
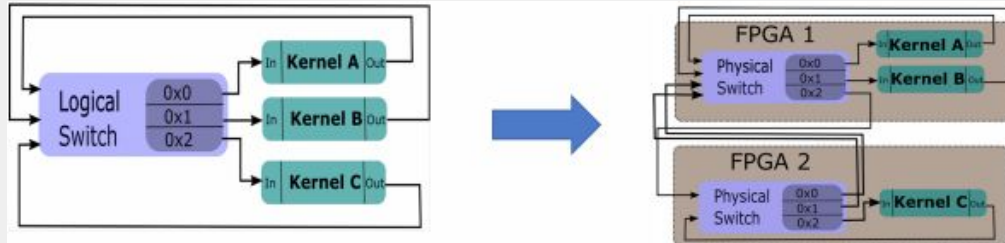
- Bobda and colleagues (2022) investigate large-scale integration of FPGAs into datacenters and cloud services
- AWS EC2 F1 and Alibaba F3 on FaaS model
- Review of different architecture and abstractions - compared useability, scalability and security

RESEARCH-ARTICLE

The Future of FPGA Acceleration in Datacenters and the Cloud

Authors: Christophe Bobda, Joel Mandebi Mbongue, Paul Chow, Mohammad Ewais, Naif Tarafdar, Juan Camilo Vega, Ken Eguro, Dirk Koch, Suranga Handagala, Miriam Leeser, Martin Herbordt, + 6

[Authors Info & Claims](#)



(a) The Shell Role Architecture principle

(b) The Mantle Architecture principle, to relax the strong dependency of the Role to low-level details of the Shell [190].

Discussion

- Can you think of a sorting algorithm which could be competitive with mergesort in hardware?
- What limits us from further speeding up the sort by partitioning the data into multiple secondary storage units, thus increasing the bandwidth?



References



Bobda, C., Mbongue, J.M., Chow, P., Ewais, M., Tarafdar, N., Vega, J.C., Eguro, K., Koch, D., Handagala, S., Leeser, M. and Herbordt, M., 2022. The future of FPGA acceleration in datacenters and the cloud. ACM Transactions on Reconfigurable Technology and Systems (TRETs), 15(3), pp.1-42.

Jun, S.W. and Xu, S., 2017, April. Terabyte sort on FPGA-accelerated flash storage. In 2017 IEEE 25th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM) (pp. 17-24). IEEE.

Koch, D. and Torresen, J., 2011, February. FPGASort: A high performance sorting architecture exploiting run-time reconfiguration on FPGAs for large problem sorting. In Proceedings of the 19th ACM/SIGDA international symposium on Field programmable gate arrays (pp. 45-54).

Qiao, W., Oh, J., Guo, L., Chang, M.C.F. and Cong, J., 2021, May. FANS: FPGA-Accelerated Near-Storage Sorting. In 2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM) (pp. 106-114). IEEE.

Srivastava, A., Chen, R., Prasanna, V.K. and Chelms, C., 2015, December. A hybrid design for high performance large-scale sorting on FPGA. In 2015 International Conference on ReConFigurable Computing and FPGAs (ReConFig) (pp. 1-6). IEEE.